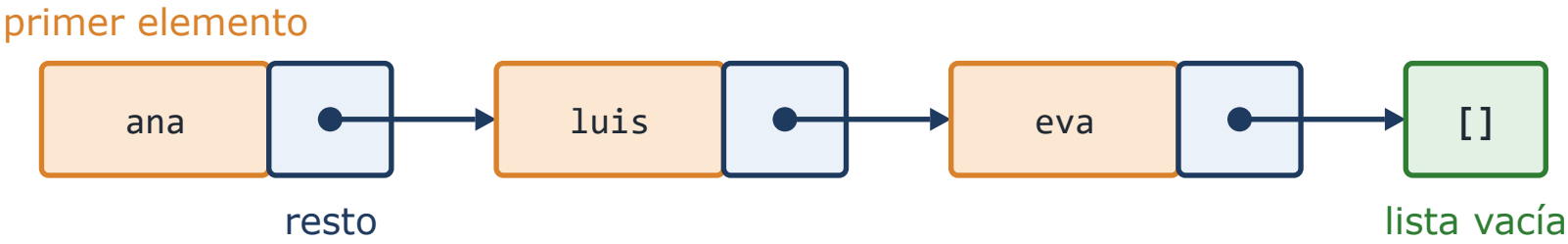
Capítulo 7 — Listas

Una lista es un término

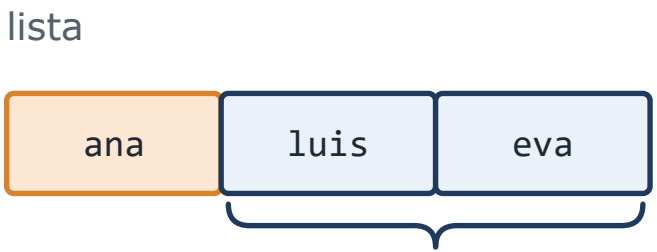
`[ana, luis, eva]`



`[ana, luis, eva]  =  [ana|[luis|[eva|[]]]]`

los corchetes evitan escribir la anidación

Primer elemento y resto



Primero	Resto
`ana`	`[luis, eva]`
`ana`	`[]`
✗ no tiene primer elemento	

La descomposición, en Prolog

```
?- [ana, luis, eva] = [Primero|Resto].
Primero = ana,
Resto = [luis, eva].

?- [ana] = [Primero|Resto].
Primero = ana,
Resto = [].

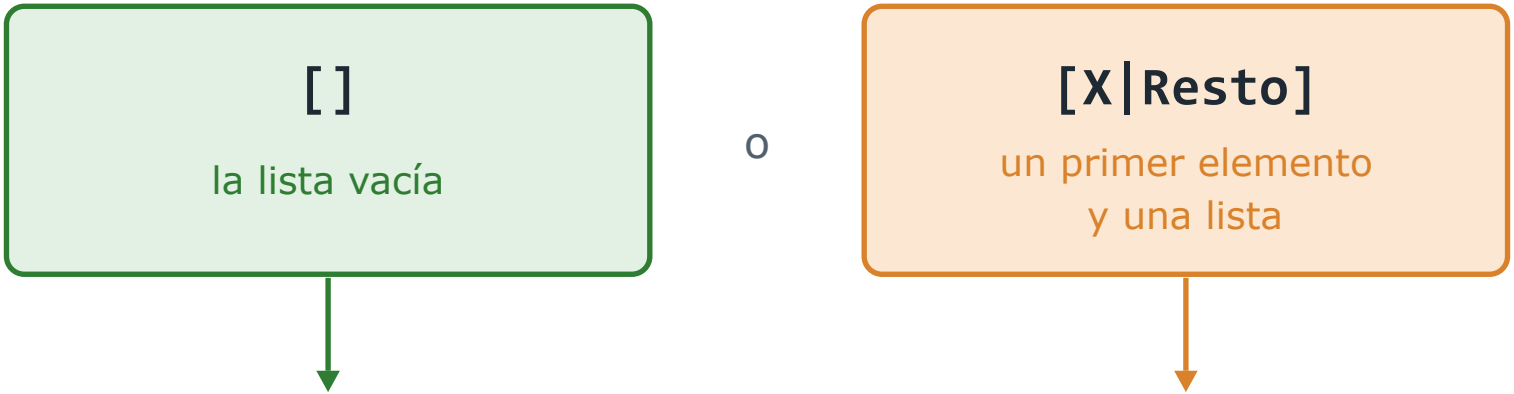
?- [] = [_|_].
false.
```

Más de un elemento al comienzo

```
?- [Primero, Segundo|Resto] = [ana, luis, eva, sofia].
Primero = ana,
Segundo = luis,
Resto = [eva, sofia].
```

Dos maneras de ser una lista

una lista es...



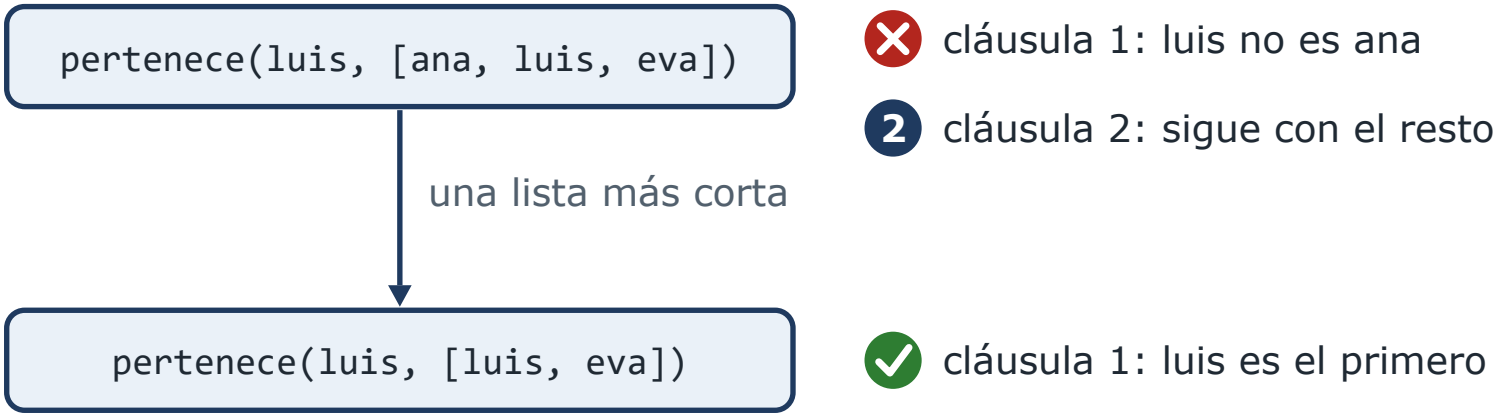

```
p([]) :- …
p([X|Resto]) :- …, p(Resto).
```

el programa reproduce la estructura que recorre

pertenece/2

```
pertenece(X, [X|_]).
pertenece(X, [_|Resto]) :-
    pertenece(X, Resto).
```

Buscar a luis




```
pertenece(X, [X|_]).
```

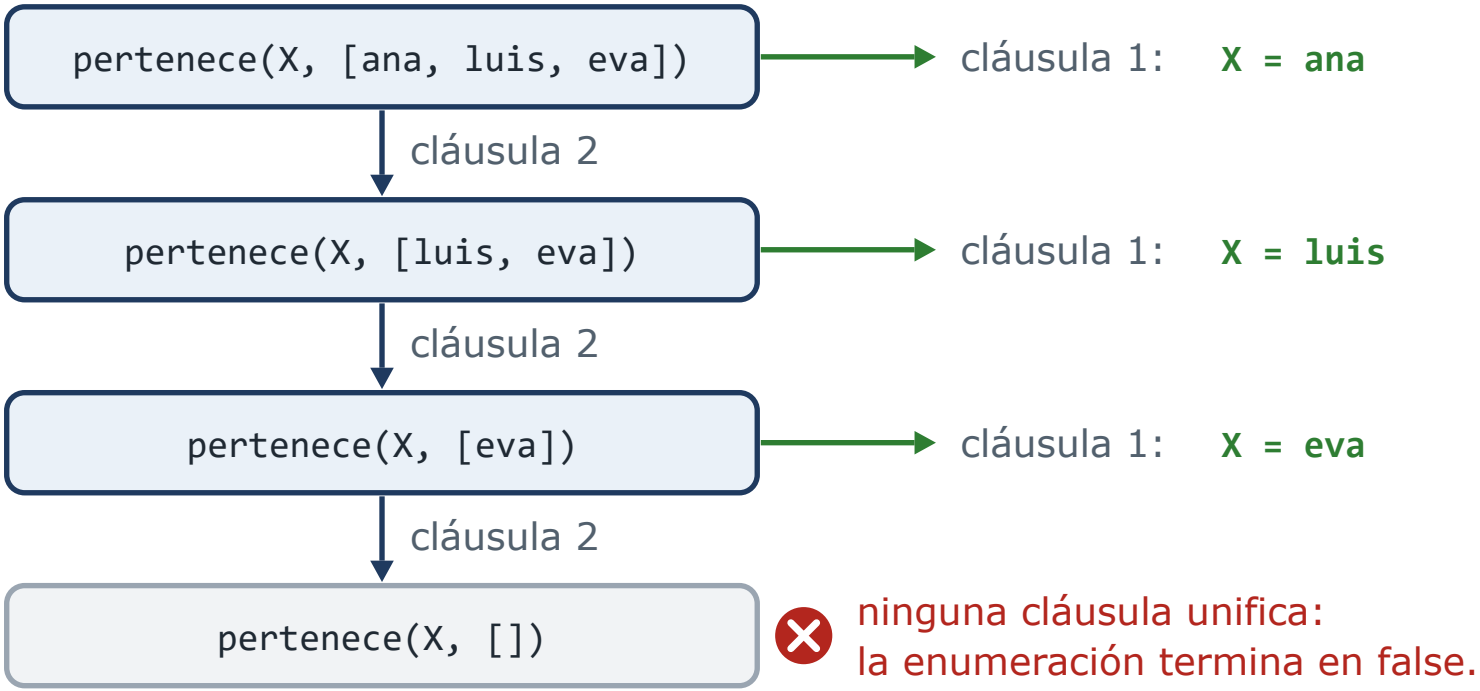
la misma variable dos veces: la unificación impone la igualdad

Verificar y enumerar

```
?- pertenece(X, [ana, luis, eva]).
X = ana ;
X = luis ;
X = eva ;
false.
```

```
?- pertenece(ana, [ana, ana]).
true ;
true ;
false.
```

Por qué termina en false



Plantilla 9 — Recorrer una lista

```
p([]) :-
    caso_de_la_lista_vacia.
```

caso base

la lista vacía

```
p([Primero|Resto]) :-
    procesar(Primero),
    p(Resto).
```

caso recursivo

procesa el primero
y sigue con el resto

A veces el caso base es un hecho; otras veces no hace falta escribirlo.

Plantilla 10 — Buscar uno que cumple

```
p([Primero|_]) :-
    cumple(Primero).
```

éxito sin recursión

en cuanto uno cumple

```
p([_|Resto]) :-
    p(Resto).
```

caso recursivo

sigue con el resto

Sobre [] no hay cláusula aplicable: ninguno cumplió.

Una respuesta por cada elemento que cumple.

todos_estan/2

```
todos_estan([], _).
todos_estan([X|Resto], L) :-
    pertenece(X, L),
    todos_estan(Resto, L).
```

```
?- todos_estan([ana, eva], [ana, luis, eva]).
true ;
false.
```

Plantilla 11 — Todos cumplen

```
p([]).
```

caso base

sobre [], «todos cumplen» es cierto

```
p([Primero|Resto]) :-
    cumple(Primero),
    p(Resto).
```

caso recursivo

prueba el primero y sigue con el resto

Fracasa en cuanto un elemento no cumple.

No produce resultado: se cumple o no.

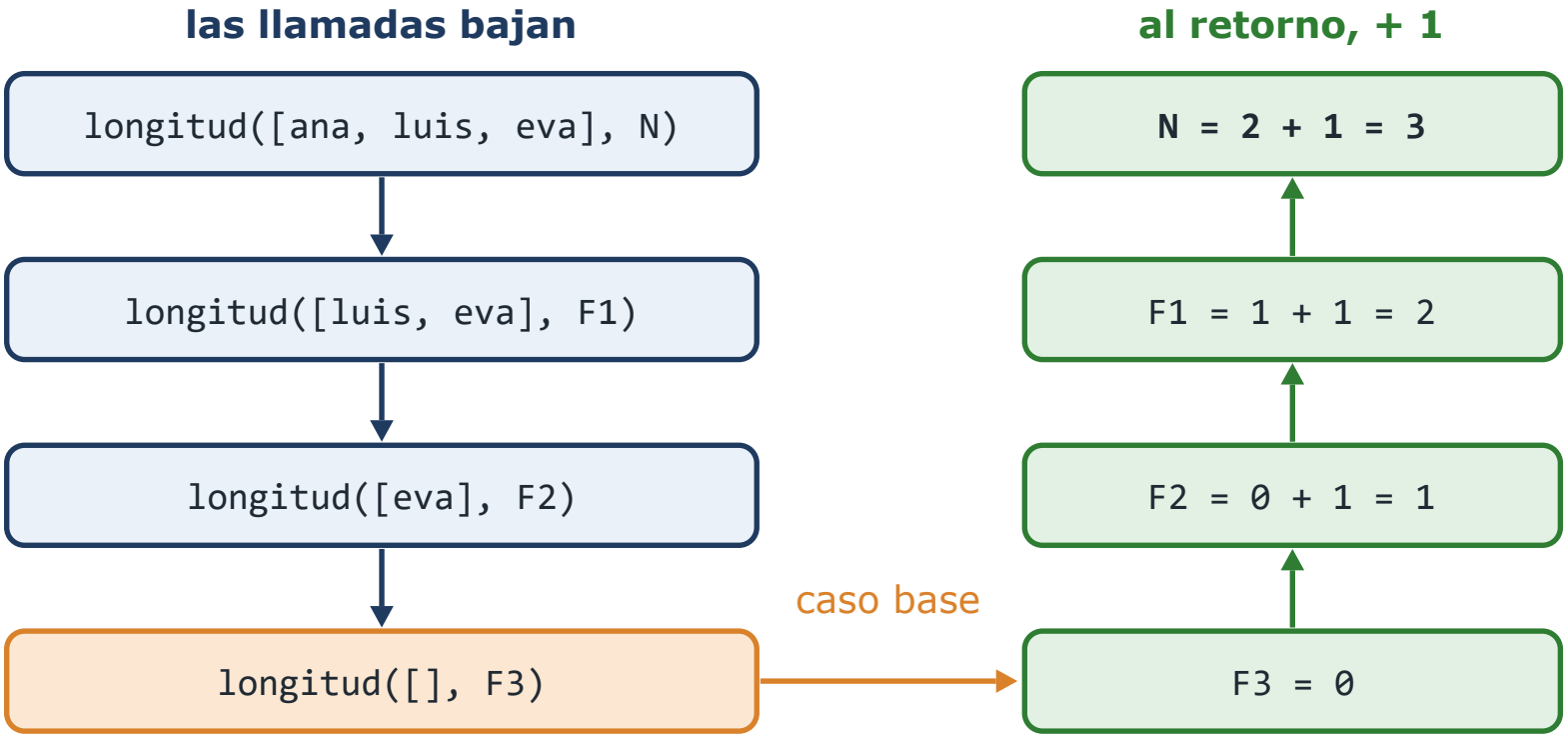
Tres siluetas

9 · recorrer	10 · buscar	11 · todos
procesa cada elemento	alguno cumple	todos cumplen
sobre []: el caso base	sobre []: fracasa	sobre []: tiene éxito
en el capítulo: **`longitud/2`**	en el capítulo: **`pertenece/2`**	en el capítulo: **`todos_estan/2`**

longitud/2

```
longitud([], 0).
longitud([_|Resto], N) :-
    longitud(Resto, Faltan),
    N is Faltan + 1.
```

El largo, al retorno

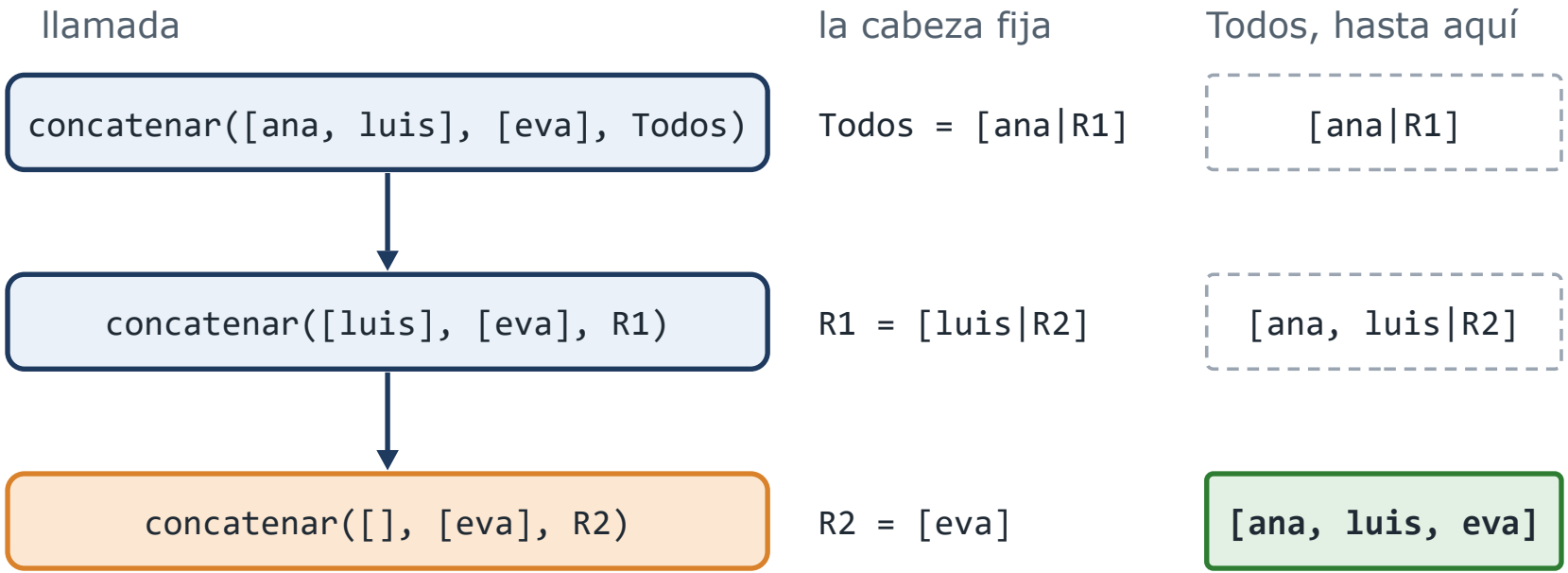


concatenar/3

```
%!  concatenar(+A, ?B, -C) is det.
%!  concatenar(?A, ?B, +C) is nondet.
%
%   C es la lista A seguida de la lista B.
concatenar([], B, B).
concatenar([X|RestoA], B, [X|RestoC]) :-
    concatenar(RestoA, B, RestoC).

?- concatenar([ana, luis], [eva], Todos).
Todos = [ana, luis, eva].
```

Construcción en la cabeza



lista incompleta en cada paso; el caso base la cierra

Plantilla 12 — Construir una lista

```
p([], CasoBase).
```

caso base

qué resultado
corresponde a []

```
p([X|Resto], [Y|RestoNuevo]) :-
    relacionar(X, Y),
    p(Resto, RestoNuevo).
```

caso recursivo

el resultado va en la cabeza:
cada llamada aporta
el primer elemento

Cambian el caso base y la relación entre cada elemento
y el que ocupa su lugar en el resultado.

Una relación, varios sentidos

```
?- concatenar(A, B, [ana, luis, eva]).
A = [],
B = [ana, luis, eva] ;
A = [ana],
B = [luis, eva] ;
A = [ana, luis],
B = [eva] ;
A = [ana, luis, eva],
B = [] ;
false.
```

Las cuatro particiones



A = []

B = [ana, luis, eva]



A = [ana]

B = [luis, eva]



A = [ana, luis]

B = [eva]



A = [ana, luis, eva]

B = []

ultimo/2

```
ultimo([X], X).
ultimo([_|Resto], X) :-
    ultimo(Resto, X).
```

```
?- ultimo([ana, luis, eva], U).
U = eva ;
false.
```

ultimo/2, en sentido inverso

```
?- ultimo(L, eva).
L = [eva] ;
L = [_, eva] ;
L = [_, _, eva] ;
...
```

El límite de los varios sentidos

predicado	termina si…	sin fin si…
`concatenar/3`	la 1.ª o la 3.ª lista está completa	la 1.ª y la 3.ª están sin determinar
`pertenece/2`	la lista está completa	la lista está sin determinar
`ultimo/2`	la lista está completa	la lista está sin determinar

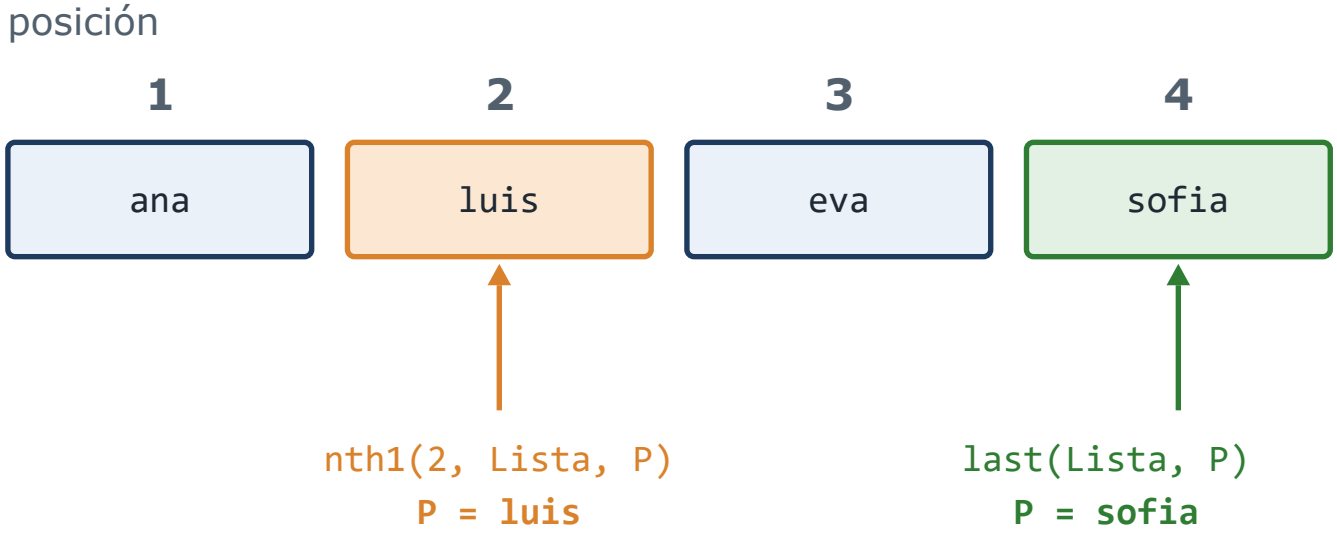
antes de un sentido nuevo: ¿qué recorre, y es finito?

Predicados predefinidos

predefinido	significa	equivale a
`member(X, L)`	X es un elemento de L	`pertenece/2`
`length(L, N)`	L tiene N elementos	`longitud/2`
`append(A, B, C)`	C es A seguida de B	`concatenar/3`
`reverse(L, R)`	R es L en orden inverso	–
`last(L, X)`	X es el último	`ultimo/2`
`nth1(N, L, X)`	X está en la posición N	–

Los invitados

```
invitados([ana, luis, eva, sofia]).
```



las posiciones se cuentan desde 1

member, length y nth1

```
esta_invitado(P) :-
    invitados(Lista),
    member(P, Lista).

cuantos(N) :-
    invitados(Lista),
    length(Lista, N).

en_el_puesto(N, P) :-
    invitados(Lista),
    nth1(N, Lista, P).
```

```
?- esta_invitado(Quien).
Quien = ana ;
Quien = luis ;
Quien = eva ;
Quien = sofia.

?- cuantos(N).
N = 4.

?- en_el_puesto(2, Quien).
Quien = luis.
```

nth1/3, en sentido inverso

```
?- en_el_puesto(N, eva).
N = 3 ;
false.
```

last y reverse

```
ultimo_en_llegar(P) :-
    invitados(Lista),
    last(Lista, P).

orden_de_salida(L) :-
    invitados(Lista),
    reverse(Lista, L).
```

```
?- ultimo_en_llegar(P).
P = sofia.

?- orden_de_salida(L).
L = [sofia, eva, luis, ana].
```

append, un invitado más

```
con_uno_mas(P, L) :-
    invitados(Lista),
    append(Lista, [P], L).

?- con_uno_mas(pedro, L).
L = [ana, luis, eva, sofia, pedro].
```

Resumen

`[]`	la lista vacía: una constante
`[X|Resto]`	primer elemento y resto, que es una lista
`[A, B|Resto]`	los dos primeros elementos y el resto
`9 · 10 · 11`	recorrer · buscar uno · todos cumplen
`12`	construir el resultado en la cabeza
`concatenar/3`	una relación: concatena y particiona
`append/3 …`	los predefinidos que usa el resto del curso

Créditos

- Ilustraciones: dibujadas para el curso, licencia MIT.
- Ejemplos: ejemplos/capitulo-07 del curso.